

Programowanie obiektowe

Wprowadzenie

- Kluczowe wyzwanie
 - Złożoność
- Rozwiązanie
 - Dekompozycja

Michał Bzowski

michal.bzowski@ing.com

www.linkedin.com/in/michalbzowski/

Najważniejsze punkty

Różnica między projektami akademickimi a rzeczywistymi systemami

- Prawdziwe systemy biznesowe mają więcej kodu, funkcji, programistów i dłuższy czas życia.
- Są utrzymywane i rozwijane przez lata, co zwiększa ich złożoność.

Złożoność jako główny problem wytwarzania oprogramowania

- W branży radzimy sobie ze złożonością przez zasadę dziel i zwyciężaj – dzielimy duży problem na mniejsze podproblemy. W paradygmatach programowania te elementy mogą przyjmować formę:
 - funkcji,
 - modułów,
 - mikroservisów,
 - obiektów (w OOP)

Dekompozycja – klucz do zrozumienia systemu

Dekompozycja pozwala analizować wybrane fragmenty systemu w izolacji.

- Umożliwia:
 - łatwiejsze rozumienie systemu,
 - ponowne użycie elementów,
 - prostsze wdrażanie nowych wymagań przez składanie istniejących części

Czego zwykle nie uczą zajęcia z OOP

- Akademickie nauczanie OOP skupia się na:
- składni,
- klasach,
- technikaliach,

...ale nie na modelowaniu problemu poprzez odpowiednie obiekty.
Brakuje tam kontekstu wielomiesięcznych zmian i rozwoju wymagań

Dlaczego OOP (a nie np. programowanie funkcyjne)?

- OOP nie jest ani „lepsze”, ani „gorsze”.
- Jest najbardziej popularne w rozwiązaniach biznesowych.
- Dziś ćwiczymy przede wszystkim szukanie właściwych abstrakcji, które można później implementować różnymi paradygmatami – OOP jest tylko wybraną formą



Kluczowa myśl

- Programowanie obiektowe
 - to **nie** nauka składni klas,
 - lecz umiejętność dzielenia złożonego problemu na właściwe, sensowne obiekty (abstrakcje), które można rozwijać w czasie.

Pierwsze zadanie

Zadanie polega na zaproponowaniu obiektów współpracujących ze sobą.

Masz przeanalizować i zaprojektować zestaw obiektów, które wspólnie obsłużą proces składania zamówienia.

- <https://github.com/michalbzowski/crc-2026-oop>
- `package ecommerce.v1_procedural.OrderProcessor;`

Pierwsze zadanie

- Istniejący kod jest jedną wielką metodą (celowo)
- Masz całkowitą swobodę w projektowaniu podziału
- Zwróć uwagę na przyszłe zmiany
- W prawdziwym systemie ten algorytm byłby mały — ale w ćwiczeniu zakładamy, że jest duży

Kluczowa myśl

Przejście od jednolitego kodu proceduralnego do zestawu obiektów o jasno określonych odpowiedzialnościach jest fundamentem myślenia obiektowego.

Celem ćwiczenia nie jest implementacja, ale modelowanie, z uwzględnieniem przyszłej ewolucji systemu.

Definicja dekompozycji algorytmicznej

- Polega na tym, że każdy krok algorytmu staje się osobnym obiektem.
- Tworzone obiekty są bezstanowe, z jedną metodą – w praktyce funkcje opakowane klasami.

Problem: Pozorna niezależność

- Choć obiekty są oddzielne, w rzeczywistości:
 - muszą być wywoływane w konkretnej kolejności,
 - są zależne od danych wyprodukowanych przez poprzednie obiekty,
 - nie działają samodzielnie i nie mogą być użyte w izolacji.

To oznacza, że nie tworzymy prawdziwych autonomicznych abstrakcji — tylko kolejne kroki algorytmu udające obiekty.

Problem: Słaba rozwijalność w czasie

- Zmiany wymagań ujawniają słabość takiego podejścia:

To klasyczny „efekt domina” — jeden krok wymaga zmian w kilku innych.

Kryterium oceny podziału: przewidywanie przyszłych zmian

Aby ocenić sensowność podziału, należy:

- antycypować zmiany (ale nie implementować ich zawczasu), używać tego jako miary — czy podział pozostanie stabilny przy zmianach?
- Jeśli najmniejsza zmiana oznacza przebudowywanie wielu obiektów → podział jest błędny.



Co oznacza „abstrakcja” w OOP?

- Abstrakcja to
 - uproszczenie rzeczywistości przez odrzucenie nieistotnych szczegółów, zależnie od kontekstu.
 - W modelowaniu obiektowym ważne są stabilne, domenowe pojęcia, a nie kroki algorytmu.

Przykład: User -> Preferencje kulinarne, wzrost, waga (chyba, że tworzymy soft dla dietetyków)

Główny problem dekompozycji algorytmicznej

- Mocne sprzężenie kolejnych kroków powoduje, że obiekty nie są autonomiczne,
- Interfejsy są skrojone pod aktualną wersję procesów, co czyni system trudnym w rozwoju i modyfikacji.

Kiedy dekompozycja algorytmiczna jest dobra?

- Gdy algorytm jest
 - stabilny
 - przewidywalny
 - raczej niezmienny.

Przykład: parsowanie plików krok po kroku (nagłówek → metadane → treść → stopka).

Kluczowy wniosek

- To, że używasz klas, nie oznacza, że programujesz obiektowo.
- Dekompozycja algorytmiczna to często tylko proceduralny kod opakowany klasami.

biektowa / domenowa / agentowa / aktorowa

- Dekompozycja obiektowa: szukanie agentów zamiast kroków algorytmu
- Zamiast traktować każdy krok jako obiekt (algorytmicznie), szukamy „aktora” w świecie rzeczywistym.
- Dane i operacje przypisujemy do obiektu, który jest ich naturalnym właścicielem.

Obiekty mają stan – i to obiekt zarządza swoimi danymi

- Getter/Setter = naruszenie enkapsulacji (najczęściej)
- „klasa danych z getterami i setterami”:
- Settery pozwalają nadpisywać stan obiektu bez kontroli.
- Gettery ujawniają szczegóły implementacji.
- To łamie zasady abstrakcji i enkapsulacji.
- W prawdziwym OOP to obiekt powinien podejmować decyzje .

Zadanie: Przeprowadź dekompozycję obiektową

- cofnij zmiany z dekompozycji algorytmicznej
- przeczytaj kod i zaproponuj aktorów, istniejących w prawdziwym świecie

Problem: pozorna niezależność – obiekty nadal są zszyte

x musi znać szczegóły y

y może potrzebować danych x.

Tworzy się cykl zależności i silne sprzężenie.

To oznacza, że obecna dekompozycja jest niedoskonała i wymaga dalszej refaktoryzacji.

Jak zmniejszyć sprzężenie?

- Szukaj stabilniejszej abstrakcji
- Zamiast:
 - `getX(techniczna reprezentacja)`,
- powinniśmy użyć:
 - `xyz()` (cecha biznesowa).

To abstrahuje od sposobu przechowywania danych i zwiększa odporność na zmiany.

Nazewnictwo powinno odzwierciedlać zamiar, a nie implementację

- Zamiast:

```
int stock = inventory.getStock(product.getId());  
int currentQuantity = getProductQuantity(product.getId());  
int maxAllowed = getMaxQuantityForUser(user, stock);  
int canAdd = Math.max(0, maxAllocwed - currentQuantity);
```

- lepiej:

```
isUserAllowedToAddProduct()
```

bo ukrywa to, jak konkretnie decydujemy o pozwoleniu

Proces nadal miesza logikę z instrukcjami warunkowymi

- Niezależność obiektów jest większa niż w podejściu algorytmicznym, ale nadal niepełna
- Obiekty potrafią wykonywać część operacji bez siebie nawzajem
- Ale ciągle nawzajem się potrzebują – co oznacza, że jakaś istotna abstrakcja została pominięta.

Gdy obiekty tworzą cykl zależności, to znak złej dekompozycji

- X potrzebuje $Y \rightarrow Y$ potrzebuje X .
- Taki „cykl” sugeruje brakujący, trzeci obiekt (np. serwis domenowy lub kontekst operacji).

Esencja w jednym zdaniu

- Dekompozycja obiektowa daje większą niezależność i stabilność niż algorytmiczna,
- ale wymaga szukania właściwych abstrakcji
- takich, które ukrywają szczegóły reprezentacji
- i eliminują wzajemne sprzężenie obiektów.

Odwiązywanie obiektów

- Zmniejszenie zależności między obiektami
- Zwiększenie rozwijalności systemu
- Usunięcie cykli zależności (x - y)

Problem wyjściowy

Obiekt X potrzebuje Y ,
a Y potrzebuje X :

- brak niezależności
- trudna wymiana lub rozbudowa
- efekt domina przy zmianach

Rozwiązanie: Koordynator

Wprowadzenie trzeciego obiektu:

Service

- bezstanowy
- koordynuje pracę obiektów
- usuwa bezpośrednie zależności

Minimalizowanie danych

Zamiast przekazywać cały obiekt:

- przekazuj tylko potrzebne dane
np. identyfikator, nazwę, flagę
- zmniejsza to sprzężenie między obiektami

Pytania kontrolne (heurystyki)

1. Czy operacja ma sens bez drugiego obiektu?
2. Czy przekazuję obiekt, czy tylko reprezentację danych?
3. Czy istnieje ukryta abstrakcja?

Przykład: Obiekt XYZ

- może grupować dane
- ale nie zawsze powinien być wymagany
- nie wymuszaj jego istnienia tam, gdzie nie ma sensu

Stabilne abstrakcje

Zastąp:

- ✗ techniczną wartość x
- ✓ wartością biznesową y

Powód: y jest stabilny, x – techniczny.

Unikanie cykli

Jeśli obiekty:

- potrzebują siebie nawzajem
- znają swoje pola prywatne
- wymagają danych z drugiego obiektu

→ to znak złej dekompozycji.

Obecność

Efekty odwiązania

- obiekty X oraz Y stają się stabilne
- zmiany dotyczą głównie koordynatora
- możliwe nowe funkcje
- system staje się skalowalny

Najważniejszy wniosek

- Jeśli widzisz cykliczne zależności między obiektami, brakuje trzeciej abstrakcji – koordynatora lub stabilnego bytu.

Rodzaje obiektów

- Rodzaje obiektów w systemie
 - Obiekty modelu
 - Obiekty koordynujące
 - Obiekty infrastrukturalne
 - Obiekty transferujące dane (DTO)
 - Obiekty wartości (Value Objects)

Obiekty modelu

- Mają **prywatny i ukryty stan**
- Podejmują decyzje biznesowe
- Modelują ważne aspekty domeny
- Abstrahują od szczegółów implementacyjnych
- Tworzą **core model aplikacji**

Obiekty koordynujące

- Reprezentują procesy biznesowe
- Łączą działania obiektów modelu
- Są **bezstanowe**
- W DDD: *application services*
- Zapewniają rozwijalność systemu

Obiekty infrastrukturalne

- Komunikacja z bazą danych
- Integracje z API
- Obsługa wejścia/wyjścia (UI)
- Oddzielają logikę biznesową od świata zewnętrznego

DTO – Data Transfer Objects

- Przenoszą dane między systemami
- Nie posiadają logiki
- Nie potrzebują enkapsulacji
- Są tylko „paczuszką danych”

Obiekty wartości (Value Objects)

- Opakowują typy proste:
 - `Price`, `ISBN`, `Email`
- Pozwalają na walidację w jednym miejscu
- Poprawiają czytelność (np. metody do arytmetyki)
- Ich wartością jest ich tożsamość

Dlaczego Value Objects?

- Brak powtarzającej się walidacji
- Łatwiejsza zmiana typu (np. float → BigDecimal)
- Lepsze modelowanie domenowe
- Większa odporność na błędy

Podsumowanie

- Różne obiekty pełnią różne role
- Obiekty modelu → serce systemu
- Koordynatory → procesy biznesowe
- Infrastruktura → komunikacja ze światem
- DTO → transport danych
- Value Objects → bezpieczeństwo + spójność

Enkapsulacja: różne jej smaki

-  Czym jest enkapsulacja?
 - Ukrywanie szczegółów implementacji
 - Obiekt jako czarna skrzynka
 - Stabilny i prosty interfejs publiczny
 - Bazuje na abstrakcji – klient nie musi znać wnętrza obiektu

Problem: getters i setters

- Ujawnia szczegóły implementacyjne
- Pozwalają klientowi wprowadzić obiekt w niepoprawny stan
- Duplikują logikę biznesową w wielu miejscach
- Zmiana typu pola = zmiana wszystkich klientów

Mutowalne struktury to zagrożenie

- Getter zwracający listę pozwala ominąć reguły
- Zamiast tego:
 - zwracaj kopię
 - używaj Value Objects
 - zapewnij tylko odczyt

Enkapsulacja zależy od typu obiektu

Obiekty modelu

- Silna enkapsulacja danych
- Reguły biznesowe wewnątrz

Koordynatory

- Enkapsulacja procesów, nie danych
- Bezstanowe

DTO

- Brak potrzeby enkapsulacji

Wyciekanie sekwencji kroków

Jeśli klient musi wywołać kilka metod w kolejności...

- ...to brak enkapsulacji
- ...algorytm wyciekł na zewnątrz
- → potrzebna jedna metoda publiczna i prywatne kroki

Getter to też wyciek wiedzy

- Getter zdradza, że w środku jest np. `int`
- Klient staje się zależny od reprezentacji
- Rozwiązanie: obiekty wartości (`Dose`, `Price`)

Dlaczego to ważne?

- Lepsza stabilność interfejsu
- Mniej zmian w przyszłości
- Większa elastyczność i odporność na zmiany
- Bardziej obiektowe podejście

Podsumowanie

- Enkapsulacja $\neq$ prywatne pola
- Gettery/settery często łamią OOP
- Uważaj na wycieki algorytmów
- Dostosuj poziom enkapsulacji do roli obiektu

Polimorfizm

- Dlaczego polimorfizm?
 - Pozwala traktować różne obiekty w jednolity sposób
 - Oddziela logikę algorytmu od wyboru konkretnego typu
 - Eliminacja instrukcji warunkowych (`if`, `switch`) w wielu miejscach

Problem bez polimorfizmu

- Algorytm musi znać wszystkie typy obiektów
- Instrukcje warunkowe mnożą się w wielu miejscach
- Zmienna logika miesza się z meritum algorytmu
- Każdy nowy typ to zmiana istniejącego kodu

Abstrakcja: Customer

- Odkryta poprzez analizę
- Różne implementacje:
 - Premium
 - Vip
 - Regular
- Koordynator komunikuje się tylko z abstrakcją

Efekt: rozszerzalność

- Dodanie nowego typu nie wymaga zmian w koordynatorze
- Zależność do stabilnej abstrakcji, nie od implementacji
- Stabilniejszy system mimo rosnącej liczby funkcji

Odkrywanie ról

User może być:

- Userem, który ma ograniczenia w liczbie produktów dodanych do koszyka
- Userem, który zostanie obciążony kwotą za zakupy
- Obiema rolami naraz

Każda rola może stać się osobną abstrakcją.

Pułapka: pochothane abstrakcje

- User i Koszyk to zbyt ogólne byty
- Różne role → różne odpowiedzialności
- Polimorfizm pomaga wydzielić właściwe abstrakcje

Gdzie wybieramy implementację?

- Na warstwie infrastruktury
- Np. poprzez repozytoria lub fabryki
- Skupiamy zmiany w jednym miejscu

Rodzaje polimorfizmu

- Przeciążanie metod
- Generyki (polimorfizm parametryczny)
- Podtypowanie (najczęstsza forma w OOP)

Polimorfizm $\neq$ obowiązek

- W prostych przypadkach lepszy jest `if`
- Polimorfizm zwiększa złożoność
- Pragmatyzm > dogmatyzm

Podsumowanie

- Polimorfizm stabilizuje zależności
- Oddziela logikę algorytmu od zmiennych typów
- Umożliwia rozszerzalność systemu
- Odkrywanie ról prowadzi do właściwych abstrakcji

Polimorfizm – implementacja i zadanie

Czym jest polimorfizm?

- Różne obiekty mogą udostępniać ten sam interfejs
- Klient nie musi znać konkretnej implementacji
- Kluczowy filar OOP obok:
abstrakcji, enkapsulacji i dziedziczenia

Polimorfizm przez interfejsy

- Deklarujemy *co* obiekt potrafi zrobić
- Implementacja pozostaje dowolna
- Obiekty implementujące interfejs mają wspólny protokół
- Popularne w: **Java, PHP, C#, C++**

Duck-typing

- Jeśli coś „ma metody jak kaczka” – może być tak użyte
- Mechanizm znany z:
Python, Ruby, JavaScript
- Polimorfizm oparty na posiadanych metodach, a nie typach

Klasy abstrakcyjne

- Dziedziczenie **zachowania i struktury**
- Klasy pochodne dziedziczą także pola
- Prowadzi to do **silnego sprzężenia**
- Zmiana w klasie bazowej → wpływa na wiele miejsc

Problem z dziedziczeniem

Przykład:

- Klasa abstrakcyjna zawiera pole `amountDue`
- 3 klasy pochodne je dziedziczą
- Jeśli jedna klasa musi zmienić typ tego pola → **zmiana dotyka wszystkie**

“ To ogranicza rozwijalność systemu.

”